

Architectural Style Justification

Lab 3: Component Modelling & Architectural Pattern Selection

Scenario: Campus Placement & Internship Pipeline (Problem Statement #03)

Name: Sonu Akash Soundararajan

SRN: PES1UG24AM908

Architecture Selection:

I chose Microservices Architecture for the Campus Placement & Internship Pipeline System.

Architectural Choice:

The system is decomposed into independently deployable services — Resume Parsing, Eligibility & Matching, Interview Queue Management, Offer & Document Processing, Notifications, and an encrypted Academic Records store — communicating over well-defined APIs and asynchronous events, rather than a single monolithic application or a purely centralized client-server model.

Two Reasons

1. Uneven, bursty load across functions: the placement cell runs hundreds of drives with overlapping deadlines, so resume parsing and eligibility filtering see extreme spikes when a drive opens, while offer processing and interview scheduling load far more gradually. Microservices let each function scale independently (e.g. horizontally scaling the Resume Parser during intake windows) instead of over-provisioning one monolith for its heaviest component.
2. Independent evolution of distinct rule sets: eligibility rules, interview workflows, and offer/document formats change per company and per drive. Isolating them into separate services lets the placement team update matching logic for one company's drive without redeploying or risking the interview-queue or offer-generation services.

Security Advantage

Sensitive data — academic records and signed offer letters — is confined to the Academic Records Data Store and the Offer & Document Processor, each of which can be hardened independently (its own AES-256 encryption-at-rest, stricter access control, and a much smaller network attack surface) without extending the same trust level to lower-sensitivity services such as the public-facing Resume Parser, which never needs direct access to encrypted offer documents.

Performance Benefit

Because each service scales on its own, the system can meet NFR-001's latency target under simulated peak load by adding Resume Parser or Eligibility Engine instances precisely where the bottleneck is, rather than scaling the entire application — which keeps infrastructure cost proportional to actual demand while avoiding the single-server bottleneck that limits throughput in a client-server design.

Campus Placement & Internship Pipeline — Component Diagram (Microservices Architecture)

○ Provided interface (ball)) Required interface (socket) □ Port > «use» dependency

